

Anteater Chess

Software Specification

Version 1.0

April 2026

Produced by: ChessEaters

EECS 22L Software Engineering Project in C Language

Henry Samueli School of Engineering

University of California, Irvine

Table of Contents

Table of Contents	1
Glossary	2
1. Software Architecture Overview	4
1.1 Main Data Types and Structures	4
1.2 Major Software Components	6
1.3 Module Interfaces	9
1.4 Overall Program Control Flow	11
2. Installation	12
2.1 System requirements	12
2.2 Setup and configuration	12
2.3 Building, compilation, and installation	12
3. Documentation of Packages, Modules, and Interfaces	13
3.1 Detailed description of data structures	13
3.2 Detailed description of functions and parameters	15
3.3 Detailed description of input and output formats	17
4. Development Plan and Timeline	19
4.1 Partitioning of Tasks	19
4.2 Team member responsibilities	19
Back matter	20

Glossary

The following terms are used throughout the software specification and implementation.

Alpha-beta pruning — An optimization technique for the Negamax search algorithm that eliminates branches which cannot affect the final move decision, significantly reducing computation time.

ANT — The enum value in PieceType representing a pawn in Anteater Chess.

ANTEATER — The enum value in PieceType represents the unique Anteater piece. Moves one step in any direction and can chain capture adjacent ANT pieces.

boardeval.h — The module responsible for evaluating a board position and returning a centipawn score relative to the side to move.

centipawn — The unit used to measure positional advantage in the board evaluation. 100 centipawns equals one pawn's worth of material advantage.

check.h — The module responsible for detecting checkmate and stalemate conditions by analyzing the current GameState.

chess.c — The main entry point of the program. Owns the game loop and coordinates all module interactions.

chess_types.h — The shared header file defining all enumerations and data structures used across every module. Has no dependencies.

Color — An enumeration with values YELLOW and BLUE representing the two players.

File — An enumeration with values A through J representing the ten columns of the 8x10 board.

GameState — The primary data structure representing the complete state of the board at a given point in time, including piece positions, turn, castling flags, en passant square, and a pointer to the previous state.

gui.h — The module responsible for graphical board display and mouse-based user interaction using SDL and X11.

Imgen.h — The module responsible for generating all legal moves for a given GameState, used by both the GUI and the AI strategy module.

logMove — A function in chess.c that writes a single move to the log file in the format: <Color><File><Rank>.-<File><Rank>.

Move — A struct representing a single move, containing a source Square (from) and destination Square (to).

MoveList — A struct containing an array of up to 250 Move structs and an integer count of valid moves in the list.

Negamax — A variant of the minimax search algorithm used by strategy.h to select the best move for the AI by recursively evaluating positions to a given depth.

Piece — A struct representing a single piece on the board, containing a PieceType and a Color.

PieceType — An enumeration of all piece types: EMPTY, KING, QUEEN, ANTEATER, BISHOP, KNIGHT, ROOK, ANT.

prev_state — A pointer field within GameState pointing to the previous board state, used to implement the undo function as a linked list of states.

Rank — An integer 1 through 8 representing a row on the 8x10 board, stored in the Square struct.

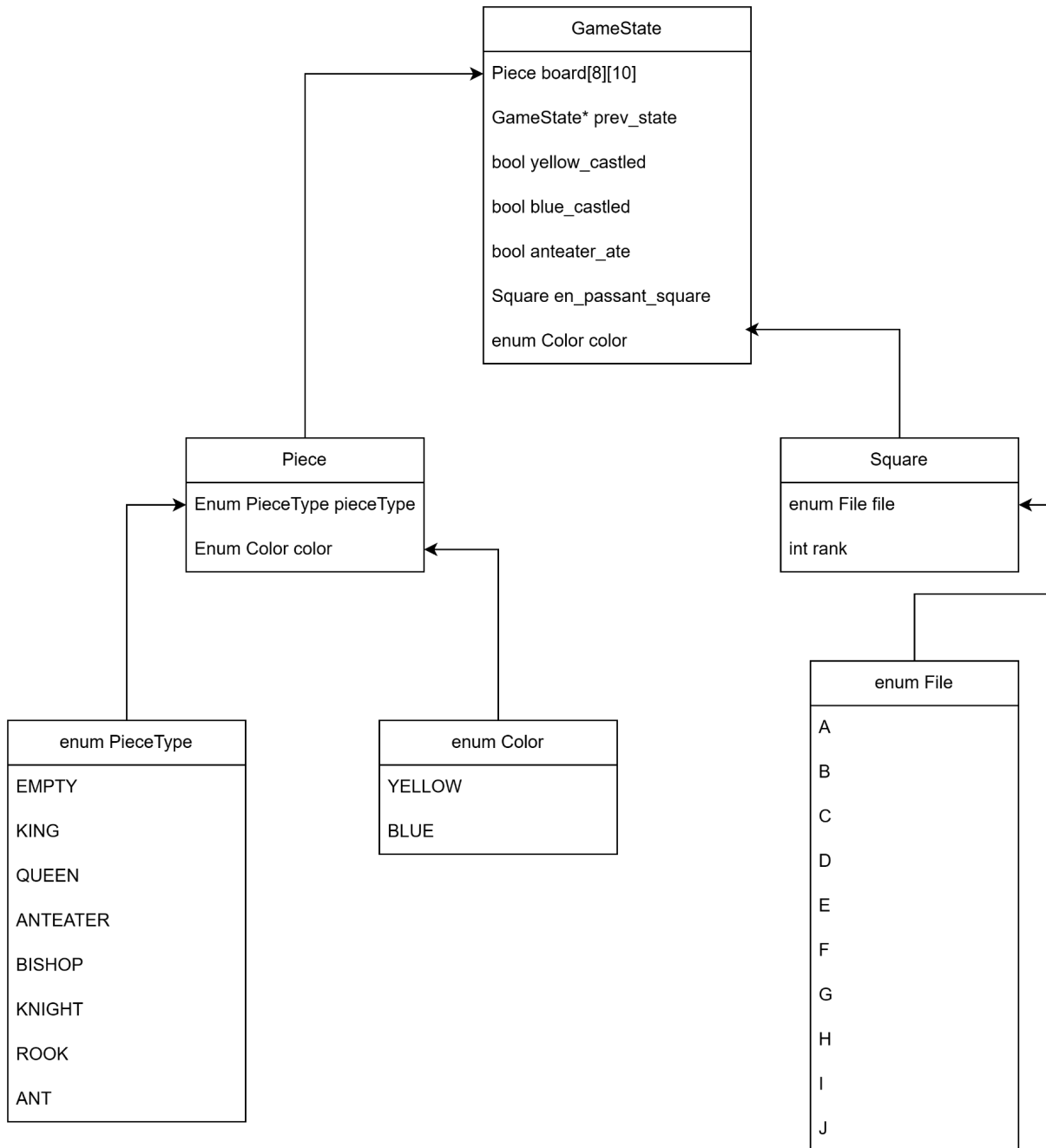
Square — A struct representing a board position, containing a File (column) and an integer rank (row).

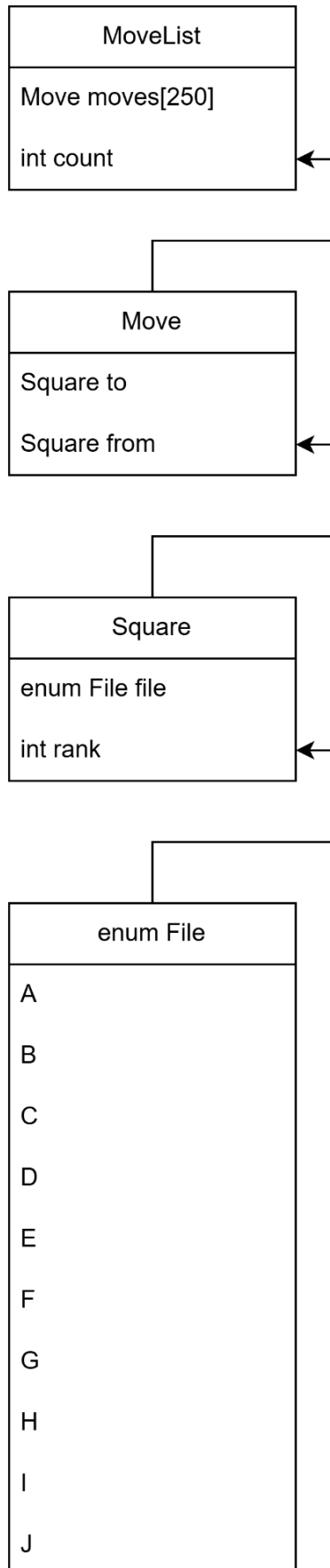
strategy.h — The module responsible for selecting the best move for the AI player using Negamax search with alpha-beta pruning at configurable depth.

textui.h — The module implementing the same interface as gui.h using text-based terminal output instead of a graphical display.

1. Software Architecture Overview

1.1 Main Data Types and Structures





Our main chessboard will be managed by a struct called GameState:

- GameState contains a Piece board[8][10] representing all 80 squares on the board
- Each array element holds a Piece struct directly, in order to simplify and prevent memory leaks
- GameState contains a GameState* prev_state pointer to the previous GameState, which handles the Undo function by maintaining a linked history of board states
- GameState contains bool white_castled and bool black_castled tracking whether each side has already used their castling move
- GameState contains bool anteater_ate tracking whether the Anteater performed a chain capture last turn
- GameState contains Square en_passant_square tracking which square is currently available for en passant capture, square struct is used for easy readability.
- GameState contains Color turn representing whose turn it is

Our pieces on the board will be managed by a struct called Piece:

- Piece is a structure representing a single piece on the board
- Contains enum PieceType representing the type of piece (KING, QUEEN, BISHOP, KNIGHT, ROOK, ANT, ANTEATER, EMPTY)
- Empty pieces will be represented by a piece of type EMPTY
- Contains enum Color representing the color of the piece (YELLOW or BLUE)

Our moves will be managed by a struct called Move:

- Move represents a single move from one square to another
- Contains Square from representing the origin square
- Contains Square to representing the destination square
- Square is struct containing rank and file, used for readability

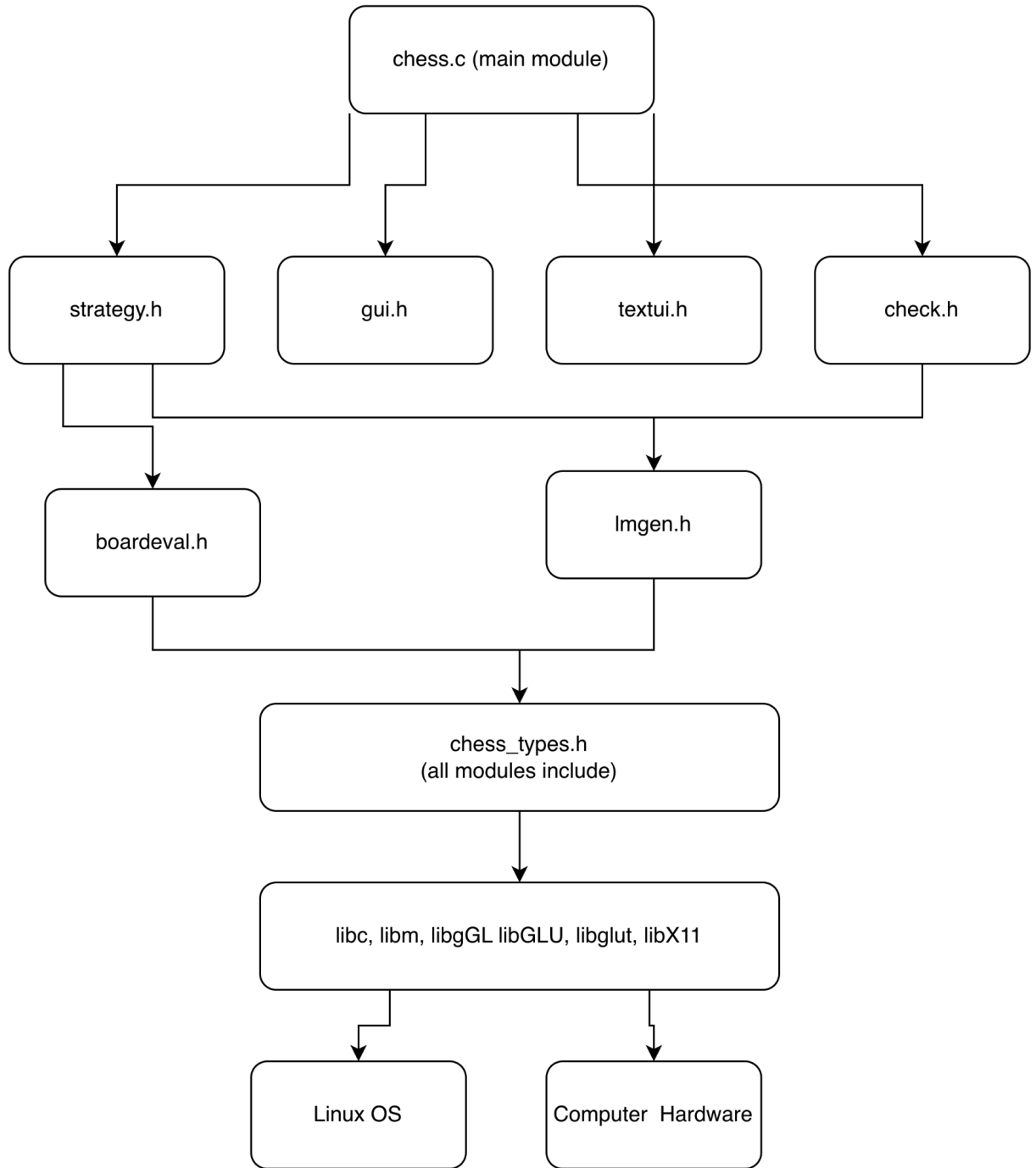
Our legal moves will be managed by a struct called MoveList:

- MoveList represents all the legal moves available in a given position
- Contains an array of Move with a maximum size of 250
- Contains int count tracking how many legal moves are on the list, needed for Strategy and GUI
- Will be initialized as part of Lmgen or find_possible_moves

1.2 Major Software Components

1. chess_types.h
 - Stores the shared definitions of all the enumerations, no dependencies
 - Included in every module
2. Lmgen.h
 - Depends on chess_types.h
 - Generates a list of all legal moves for all piece types belonging to player
3. Check.h
 - Dependencies include chess_type.h and lmgen.h
 - Will analyze checkmate for both given players at conclusion of previous turn
4. boardeval.h
 - Depends on chess_types.h
 - Takes the current chess board state and returns centipawn score representing who is winning and by how much
5. strategy.h
 - Depends on chess_types.h, lmgen.h, boardeval.h
 - Selects the best move for the AI player using Negamax search with alpha beta pruning.
6. gui.h

- Depends on chess_types.h, lmg.h, strategy.h
- 7. textui.h
 - Depends on chess_types.h, lmg.h, strategy.h
- 8. chess.c
 - Depends on all of the above
 - Entry point of program



1.3 Module Interfaces

chess_types.h

- No functions just types

Imgen.h

- AI Strategy module will call a function to generate legal moves
- GUI will call a function when a piece is clicked to do path projection on the screen and to prevent illegal moves.

check.h

- Will analyze King piectype on current GameState gs for current checkmate status
- GUI will call function and when return is true, checkmate will be declared

boardeval.h

```
int evaluate(struct GameState gs)
```

- Master evaluation function that can be called for the negamax AI algorithm. This function will sum up all the terms and negate it for the algorithm.

strategy.h

```
Move* SelectBestMove(Gamestate *gs, Color color, int depth)
```

- Returns the best move for the AI player given the current game state and search depth

gui.h/textui.h

```
void displayBoard(GameState *gs, int yellowSecs, int BlueSecs, Color humanColor)
```

- Displays the board graphically in GUI

```
void dispLegalMoves(MoveList *moves)
```

- Visually highlights the pathways for a selected piece to move

```
Move getMove(GameState *gs)
```

- Waits for the player to click a piece and select a destination on the visualized board

```
Void dispWin(Color winner)
```

- Visually displays the winning piece and which color won

```
int matchupMenu (void)
```

- Screen of the matchup: user vs. user, user vs. AI, or AI vs. AI

textui.h

- Implements the same interface as gui.h using text based terminal output instead of SDL/X11.

chess.c

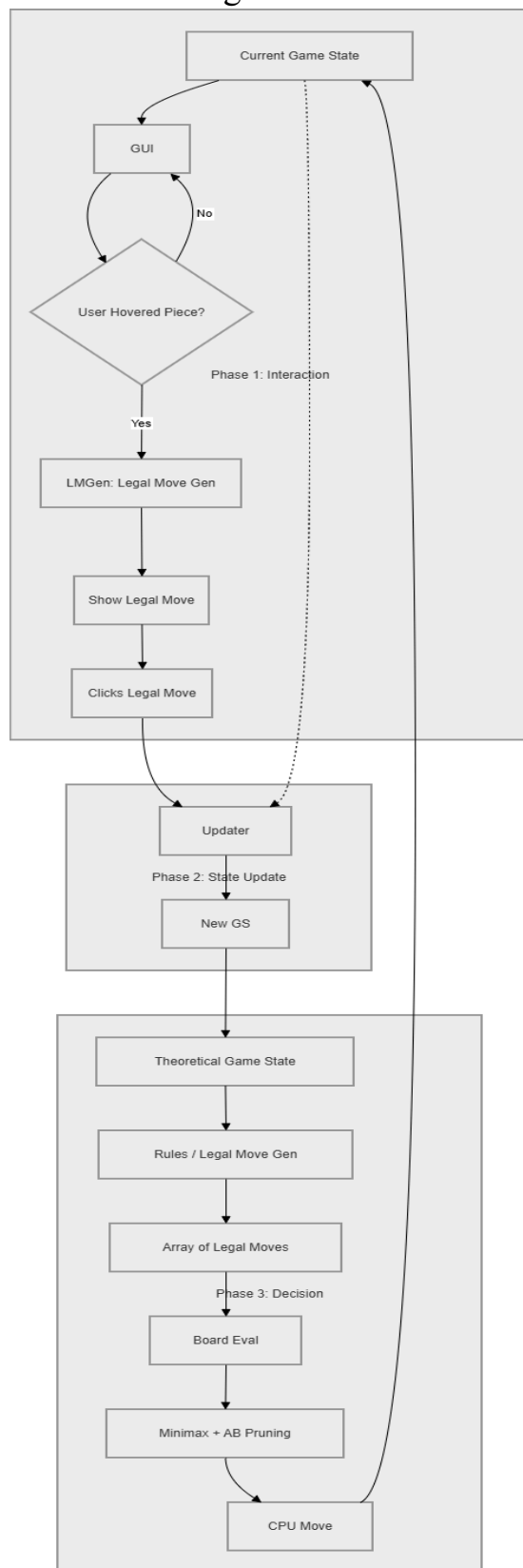
```
int main(void)
```

- Entry point of the program. Initializes game state, launches matchup menu, and runs the main game loop.

```
void logMove(FILE *logfile, Color color, Move move)
```

- Writes one move to the log file in the format: <Color> <File><Rank>-<File><Rank>

1.4 Overall Program Control Flow



2. Installation

2.1 System requirements

The following are required to run Anteater chess.

- Operating system
- Linux (tested on UCI EECS servers: Bondi, Laguna, Crystalcove, Zuma, Balboa, Doheny)
- GCC compiler (version 9.0 or later)
- GNU Make
- SDL (for GUI display) pre installed on UCI servers
- X11 display (required for GUI display)
- Minimum 512 MB of RAM, 50 MB of free disk space

2.2 Setup and configuration

To install and run Anteater chess from the binary package, download the file and run the following commands.

- `gtar xvzf Chess_V1.0.tar.gz`
- `chess/bin/chess`

To build from source code, download the source package and run the following commands

- `gtar xvzf Chess_V1.0_src.tar.gz`
- `cd chess`
- `make`
- `chess/bin/chess`

2.3 Building, compilation, and installation

To compile from source, navigate to the chess directory and run:

- `make`

The Makefile compiles and links the following modules:

- `chess.c`
- `lmgcn.c`
- `strategy.c`
- `boardeval.c`
- `gui.c`
- `textui.c`

Required compiler flags

- `gcc -o chess/bin/chess chess.c lmgcn.c strategy.c boardeval.c gui.c textui.c -lm -lGL -lGLU -lglut -lX11`

To remove compiled objects and executable:

- `make clean`

3. Documentation of Packages, Modules, and Interfaces

3.1 Detailed description of data structures

Chess_types.h

```
typedef enum {  
    A,  
    B,  
    C,  
    D,  
    E,  
    F,  
    G,  
    H,  
    I,  
    J  
} File;
```

```
typedef enum {  
    EMPTY,  
    KING,  
    QUEEN,  
    ANTEATER,  
    BISHOP,  
    KNIGHT,  
    ROOK,  
    ANT  
} PieceType;
```

```
typedef enum {  
    YELLOW,  
    BLUE  
} Color;
```

```
typedef struct {  
    File file;  
    int rank;  
} Square;
```

```
typedef struct {
    PieceType piecetype;
    Color color;
} Piece;
```

```
typedef struct {
    Square from;
    Square to;
} Move;
```

```
typedef struct {
    Move moves[250];
    int count;
} MoveList;
```

```
typedef struct GameState {
    Piece board[8][10];
    struct GameState* prev_state;
    bool yellow_castled;
    bool blue_castled;
    bool anteater_ate;
    Square en_passant_square;
    Color turn;
} GameState;
```

GameState stores a pointer prev_state to the previous board state allowing the program to revert moves for the undo function.

lmggen.h

- No new data structures defined
- Depends on chess_types.h
- Uses GameState, MoveList, Move, Square, Piece, and Color from chess_types.h

check.h

- No new data structures defined
- Depends on chess_types.h and lmggen.h
- Uses GameState and Piece from chess_types.h

boardeval.h

- Uses GameState, PieceType, Piece, Color, Square and File from chess_types.h

strategy.h

- No new data structures defined
- Uses gamestate, MoveList, Move, and Color from chess_types.h

gui.h

- No new data structures defined
- Depends on chess_types.h, lmggen.h, and strategy.h
- Uses GameState, MoveList, Move, and Color from chess_types.h

textui.h

- No new data structures defined
- Depends on chess_types.h
- Uses GameState, MoveList, Move, and Color

chess.c

- Depends on all of the above

3.2 Detailed description of functions and parameters

Lmgen.h

```
struct MoveList Lmgen(const struct *GameState gs)
```

Arguments

- gs - pointer to a game state to analyze

Result:

- Return value MoveList- struct containing a list of legal moves and a length of that list.

Notes:

- To be used for the strategy module, and GUI

```
struct MoveList find_possible_moves(const struct *GameState gs, struct Square square )
```

Arguments

- gs - pointer to a game state to analyze
 - When Color color transitions find_possible_moves will execute
- square- pointer to a the squares in that game state that contains a piece to generate legal moves for

Result:

- Return value MoveList- struct containing a list of legal moves and a length of that list for a piece on the current square

Notes:

- To be used to project paths if a user clicks any piece on the GUI
- Will run at the conclusion of previous colors turn

check.h

```
bool checkmate(const struct GameState *gs, struct Piece Piecetype)
```

Argument

- Gs - pointer to the current turns Gamestate
- Piece Piecetype - analyze for the Piecetype KING for both Color enums

Result:

- Return value will be boolean, True if checkmate, and False otherwise

Notes

- Will be used by GUI to declare victory

strategy.h

```
Move* SelectBestMove(GameState *gs, Color color, int depth)
```

Arguments:

- Gs - pointer to the current gamestate
- Color - the ai players color (YELLOW or BLUE)
- Depth - search depth determined by difficulty (Easy = 2, Medium = 4, Hard = 6).

Result:

- bestMove - pointer to the highest scoring move found by Negamax with alpha beta pruning.

Notes:

- Returns NULL if no legal moves are available(checkmate or statemate)

Boardeval.h

```
int evaluate (GameState *gs)
```

Arguments:

- Gs - pointer to the current gamestate

Results:

- Returns centipawn integer score relative to the side to move (gs→turn)
- Positive value = side currently to move is winning
- Negative value = side currently to move is losing
- Zero value = position is roughly equal

Notes:

- To be used for the strategy module

Gui.h

```
void displayBoard(GameState *gs, int yellowSecs, int blueSecs, Color humanColor)
```

Arguments:

- Gs -pointer to current game state
- yellowSecs - integer representing remaining time in sec for yellow player
- blueSecs - integer representing remaining time in sec for blue player
- humanColor - Color representing the human player's color to orient to board

Result:

- No return value. The graphical board with all the pieces and clocks are displayed.

Notes:

- Called by chess.c every time the board needs to be redrawn.

```
void dispLegalMoves(MovesList *moves)
```

Arguments:

- Moves - pointer to MoveList generated by find_piece_moves in lmg.h

Result:

- No return value. Highlights all valid destination squares on the board

Notes:

- Called by chess.c during human turn after a piece is clicked

```
Move getMove(GameState *gs)
```

Arguments:

- Gs - pointer to current game state

Result:

- Return value Move - struct containing the selected Square from and Square to

Notes:

- Called by chess.c during human turn to get the player's move

```
int matchupMenu(void)
```

Arguments:

- None

Results:

- Return value int representing the selected matchup

Notes:

- Called by chess.c at games start
- Options are user vs user, user vs ai, or ai vs ai

```
void dispWin (Color winner)
```

Arguments:

- Winner - Color representing winning player

Result:

- No return value. Displays victory message on screen for the winning color.

Notes:

- Called by chess.c when checkmate is detected by check.h
- dispStalemate and dispTimeout follow the same pattern for draw and timeout conditions

Textui.h

- Implements the same interfaces as gui.h using text-based terminal output instead of GUI. See gui.h above for function prototypes and parameters.

Chess.c

Chess.c is the main entry point for the program and controls the overall game loop.

```
Void logMove(FILE *logfile, Color color, Move move)
```

Arguments:

- Logfile - pointer to open log file
- Color - the Color of the player who made the move
- Move - the move that was executed

Result:

- No return value. Writes one line to the log file in the format: <Color> <File><Rank> - <File><Rank>

3.3 Detailed description of input and output formats

- Syntax or format of move entered by the user

<File><Rank> <File><Rank>

File will be single letter A-J (column of board), Rank is single digit 1-8 (row of board). The origin piece in a square is first, then the coordinates of the destination in the second part separated by a single space.

- Syntax or format of a move recorded in the log file

<Color> <File><Rank>-<File><Rank>

Color is going to show either yellow or blue, like the syntax/format entered by the user, the origin and destination squares are separated by a dash instead of a space.

4. Development Plan and Timeline

4.1 Partitioning of Tasks

Development is divided into two phases: alpha and beta.

In the alpha phase, each team member focuses on implementing the core functionality of their assigned module. The priority order is `chess_types.h` first (no dependencies), followed by `lmg.h` and `boardeval.h` in parallel, then `check.h`, `strategy.h`, and `textui.h` once `lmg.h` is functional. `gui.h` and `chess.c` are integrated last once all other modules have working prototypes.

In the beta phase, team members whose alpha deliverables are complete will assist with GUI integration, end to end testing, and bug fixes across modules. Testing will be conducted incrementally, first verifying legal move generation, then AI move selection, then full game loop functionality including checkmate detection, stalemate, and timeout.

4.2 Team member responsibilities

- Collan Harrington - `strategy.h/strategy.c/chess.c`
- Claire Nguyen and Tara Pai - `LMGen.h/LMGen.c/Check.h`
- Jordan Le and Aurea Jolly - `textui.h/gui.h`
- Yanilette Montano - `boardeval.h/boardeval.c`

Back matter

COPYRIGHT

© 2026 ChessEaters, University of California, Irvine. All rights reserved. This software was developed as a course project for EECS 22L. Redistribution and use in source and binary forms, with or without modification, are permitted for educational purposes only.

INDEX

AI strategy module§1.2, §1.3, §3.2	GameState§1.1, §3.1
Alpha-beta pruning§1.3, §3.2	GUI module§1.2, §1.3, §3.2
Board evaluation§1.2, §1.3, §3.2	Legal move generator§1.2, §1.3, §3.2
boardeval.h§1.2, §1.3, §3.2	Imgen.h§1.2, §1.3, §3.2
check.h§1.2, §1.3, §3.1	Move§1.1, §3.1
chess.c§1.2, §1.3, §3.2	MoveList§1.1, §3.1
chess_types.h§1.2, §3.1	Negamax§1.3, §3.2
Color selection§3.3	strategy.h§1.2, §1.3, §3.2
Compilation flags§2.3	System requirements§2.1
Control flow§1.4	Text UI module§1.2, §1.3, §3.3
Development timeline§4.1, §4.2	

REFERENCES

- [1] *Chess.com*. Rules of Chess. <https://www.chess.com/learn-how-to-play-chess>. Accessed April 2026.
- [2] *UCI EECS 22L Course Staff*. Anteater Chess Project Specification. University of California, Irvine, 2026.
- [3] *Chessprogramming Wiki*. Negamax. <https://www.chessprogramming.org/Negamax>. Accessed April 2026.